

Gestión de Estados de Carga en aplicación de Control de Asistencia

En el diseño de sistemas interactivos, la comunicación constante con el usuario final es un principio fundamental. Cuando una aplicación cliente (frontend) necesita solicitar datos a un servidor externo (API) —como el listado de estudiantes matriculados—, esta operación no ocurre de forma instantánea. Depende de la latencia de la red y el tiempo de procesamiento del servidor.

Para garantizar una experiencia de usuario fluida y evitar la incertidumbre, la arquitectura del sistema implementa un **patrón de gestión de estados**. Este patrón transiciona la interfaz gráfica a través de tres fases mutuamente excluyentes:

1. **Carga (Loading):** Informa que hay un proceso en segundo plano recolectando la información.
2. **Éxito (Data/Success):** Renderiza la información solicitada cuando el proceso de red culmina correctamente.
3. **Error:** Captura excepciones (caídas de red, fallos del servidor) y ofrece mecanismos de recuperación.

El manejo explícito de estos estados previene que la aplicación parezca "congelada", evita el envío de solicitudes duplicadas por parte del usuario y provee robustez frente a fallos, cumpliendo con los estándares de calidad de software requeridos para un producto funcional.

```

50 // Consumo de API (fetch) y cambio de estados
51 async function fetchStudents() {
52   state.loading = true;
53   state.error = null;
54   renderUI();
55
56   try {
57     if (state.simulateError) {
58       throw new Error('Error simulado: No se logró conectar con el servidor de matrículas de la Univ');
59     }
60
61     const response = await fetch(API_URL);
62
63     if (!response.ok) {
64       throw new Error(`HTTP Error status: ${response.status}`);
65     }
66
67     const rawUsers = await response.json();
68
69     state.students = rawUsers.map(user => ({
70       id: user.id,
71       name: user.name,
72       code: `EST-${202600 + user.id}`,
73       email: user.email.toLowerCase(),
74       phone: user.phone.split(' ')[0],
75       status: 'presente'
76     }));
77
78     state.loading = false;
79   } catch (err) {
80     state.loading = false;
81     state.error = err.message || 'Ocurrió un error inesperado al consultar la API.';
82   }
83

```

Esta captura expone la lógica asíncrona de la aplicación a través de la función `fetchStudents()`. Se observa la implementación de un bloque `try/catch` que encapsula el consumo de la API web.

Aquí se manipula el estado de forma directa. Antes de iniciar la petición de red, se establece `state.loading = true` y se limpia cualquier error previo. Dentro del `try`, destaca la inclusión de una validación estructurada para simular un error forzado; este tipo de mecanismos es vital en etapas de pruebas de calidad (QA) para evaluar el comportamiento del sistema bajo fallos. Si la promesa de red es exitosa, se mapean los datos, se cambia el estado de carga a `false` y se invoca la actualización de la interfaz. Si ocurre un fallo, el bloque `catch` lo intercepta, detiene el estado de carga e inyecta el mensaje exacto del fallo en `state.error`.

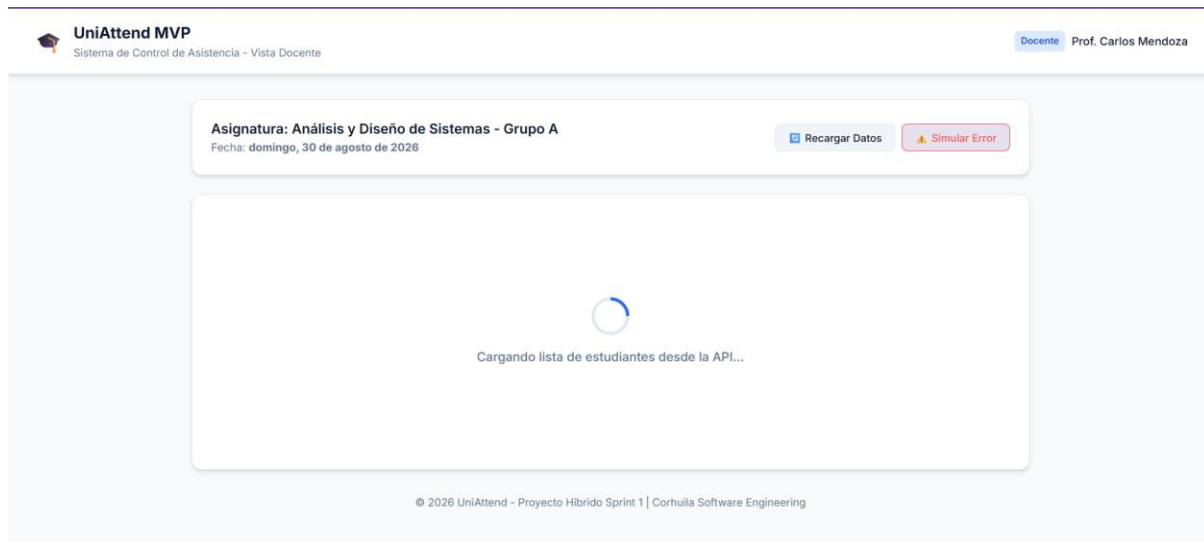
```

87 // Controlador de Vista (Manejo de Estados)
88 function renderUI() {
89     // 1. Ocultar todos los contenedores
90     stateLoading.classList.add('hidden');
91     stateError.classList.add('hidden');
92     stateData.classList.add('hidden');
93     metricsSummary.classList.add('hidden');
94
95     // 2. Mostrar la vista correspondiente según el estado actual
96     if (state.loading) {
97         stateLoading.classList.remove('hidden');
98     } else if (state.error) {
99         errorMessage.textContent = state.error;
100        stateError.classList.remove('hidden');
101    } else {
102        renderStudentTable();
103        updateMetrics();
104        stateData.classList.remove('hidden');
105        metricsSummary.classList.remove('hidden');
106    }
107 }
108
109 function renderStudentTable() {
110     studentList.innerHTML = '';
111     state.students.forEach(student => {
112         const tr = document.createElement('tr');
113         const initials = student.name.split(' ').map(n => n[0]).slice(0, 2).join('');
114
115         tr.innerHTML = `
116             <td>
117                 <div class="student-info">
118                     <div class="avatar">${initials}</div>
119                     <strong>${student.name}</strong>
120                 </div>

```

La imagen muestra el núcleo del controlador de la vista: la función `renderUI()`. Este bloque de código actúa como un "director" unificado para el DOM (Document Object Model).

En lugar de modificar el estilo de cada elemento de forma dispersa, el sistema utiliza un enfoque centralizado. Primero, la función resetea la vista ocultando todos los contenedores principales (inyectando la clase CSS `.hidden`). Posteriormente, evalúa las variables del objeto global `state` (`state.loading` y `state.error`) mediante una estructura condicional. Dependiendo de los valores, remueve la clase `.hidden` únicamente del contenedor que corresponde al estado actual. Esto garantiza modularidad y asegura que nunca se superpongan dos pantallas (por ejemplo, mostrar un error y la tabla al mismo tiempo).



Esta captura evidencia la vista que se le presenta al docente en el momento exacto en que el sistema solicita los datos al servidor. A nivel de Interfaz de Usuario (UI), se oculta la tabla principal y se despliega un indicador visual de progreso (*spinner* circular) acompañado del mensaje "Cargando lista de estudiantes desde la API...".

Su propósito principal es retroalimentar al usuario de que su solicitud fue recibida y se está procesando. Desde el punto de vista de la ingeniería del software, este estado bloquea implícitamente interacciones prematuras con la tabla de asistencia hasta que el conjunto de datos esté íntegro y listo para ser renderizado, asegurando la consistencia de la información.

Asignatura: Análisis y Diseño de Sistemas - Grupo A

Fecha: domingo, 30 de agosto de 2026

Recargar Datos

Modo Normal



Error al cargar la información

Error simulado: No se logró conectar con el servidor de matrículas de la Universidad.

Reintentar Conexión

La imagen final ilustra la interfaz de contingencia (Fallback UI). Tras activar la simulación de error, el sistema detecta la falla, oculta el indicador de carga y presenta un panel de alerta claro con un ícono de rechazo.

Este estado cumple una doble función crítica en el análisis y diseño del sistema. Primero, informa al docente de manera transparente sobre el motivo del fallo ("Error simulado: No se logró conectar con el servidor de matrículas..."), evitando frustración. Segundo, implementa un mecanismo de recuperación proactivo mediante el botón "Reintentar Conexión". A nivel de arquitectura, proveer alternativas para que el usuario recupere el flujo de trabajo sin necesidad de reiniciar completamente la aplicación es una práctica esencial en el diseño de sistemas tolerantes a fallos.

La correcta implementación de estos estados visuales y lógicos garantiza que UniAttend MVP ofrezca una experiencia de usuario transparente y robusta. Al gestionar la asincronía de los datos y prever escenarios de fallo, el sistema no solo mejora la interacción del docente, sino que también cumple con los estándares técnicos fundamentales en la ingeniería de software moderna para crear aplicaciones confiables y tolerantes a errores.